i need you to create a very detailed documentation for the whole project which...

Below is a full, end-to-end documentation draft for your MVP. You can treat this as the “master document” for your project: product doc, functional spec, and high-level technical design all in one.

You can copy this into Confluence / Notion / Google Docs and adapt as your project evolves.

AI-Powered Recruitment System – MVP Documentation

1. Introduction

1.1 Purpose of This Document

This document describes the **design, behavior, and usage** of the AI-powered recruitment MVP. It consolidates:

- Product overview and objectives
- Detailed functional requirements
- System architecture and technical design
- Data flows and interactions
- User interfaces and workflows
- Deployment, configuration, and operations guidance

It is intended for:

- Product Owners and Stakeholders
- Engineering and QA Teams
- UX/UI Designers
- DevOps / Infrastructure Engineers

🚀 Powered by Yupp

2. Product Overview

2.1 Problem Statement

Recruiting processes are often slow and manually intensive. Typical bottlenecks:

- Manually reading through 100–1000+ resumes per role
- Conducting many first-round interviews that add limited value
- Non-standard evaluation and lack of objective comparison across candidates

This MVP aims to **automate early-stage screening and improve first-round interviews** using AI.

2.2 Solution Summary

The AI-powered recruitment MVP provides:

1. **Resume Screening Module**
   - Extracts evaluation criteria from job descriptions.
   - Screens and scores resumes against configured criteria.
   - Ranks candidates and surfaces the top ones for interviews.
2. **Interview Assistance Module**
   - Captures audio (single microphone) of interviewer + candidate.
   - Performs real-time transcription with speaker diarization.
   - Provides real-time assistance and follow-up question suggestions.
   - Generates a post-interview scorecard based on conversation.
3. **Final Candidate Evaluation Engine**
   - Aggregates:
     - Resume score
     - Interview performance
     - Interviewer feedback
   - Produces a final candidate score for decision-making.

---

3. Project Objectives

The MVP aims to:

🚀 Powered by Yupp

- **Automate resume screening** to quickly identify strong candidates.
- **Assist interviewers** with real-time insights to conduct more structured and effective interviews.
- **Standardize candidate evaluation** via scorecards and final scoring that reflect job requirements and interviewer feedback.
- **Reduce time-to-shortlist** by at least 50% compared to manual screening.

---

4. Scope of Work (Functional Overview)

4.1 Modules

The MVP consists of three core functional modules:

1. **Resume Screening Module**
2. **Interview Assistance Module**
3. **Final Candidate Evaluation Engine**

Each module interacts with shared storage (database, file storage) and external AI services.

---

5. Functional Specifications

5.1 Resume Screening Module

5.1.1 Job Description Processing

Goal: Convert a free-text job description into structured, configurable evaluation criteria.

Input:

- Job description (text, typically pasted or uploaded from JD source)
- Optional metadata:
  - Job title
  - Department
  - Seniority level

Processing Flow:

1. **User Action:**

🚀 Powered by Yupp

- User creates a new job in the UI and inputs/pastes the job description.

2. **LLM Processing:**
   - The system invokes the LLM (e.g., Gemini 2.0 Flash or equivalent) with a prompt including:
     - The raw job description text.
     - Instruction to output structured criteria in a standardized JSON schema.
   - The LLM extracts criteria such as:
     - Required experience level (e.g., "3–5 years in backend development").
     - Salary expectations (if mentioned).
     - Location requirements (e.g., "USA-based, remote OK").
     - Technical skills and competencies (e.g., "Python, React, Kubernetes").
     - Educational qualifications (e.g., "Bachelor's degree in CS or related field").
     - Industry-specific requirements.
3. **Standardized Criteria Schema:**
   Typical internal representation for each job may include:
   - `experience_years_min`, `experience_years_max`
   - `salary_range_min`, `salary_range_max` (optional)
   - `location_requirements` (country, city, on-site/remote/hybrid)
   - `skills` (each skill with importance level)
   - `education` (minimum degree, preferred degrees)
   - `other_requirements` (certifications, domain experience, etc.)
4. **Validation:**
   - System performs basic validation:
     - Required fields present?
     - Types correct (numbers, strings, lists)?
   - If something fails, default values or manual user input will be used.

Output:

- A structured "Job Criteria Config" object stored in the database.
- This becomes the default evaluation template for that job.

5.1.2 Criteria Threshold Configuration

Goal: Allow hiring stakeholders to refine and configure criteria.

🚀 Powered by Yupp

User Actions:

- **Review AI-generated criteria** in a configuration UI.
- **Set minimum thresholds**, such as:
  - `experience_years_min = 4`
  - Required skills must have a minimum proficiency level.
- **Adjust weights / priorities**, such as:
  - `Skill: Python, weight = 0.3`
  - `Experience length, weight = 0.25`
  - `Education level, weight = 0.15`
  - `Domain experience, weight = 0.2`
  - `Location fit, weight = 0.1`

Functional Behavior:

- The UI displays each criterion:
  - Name
  - Description / rationale (optional explanation from the LLM)
  - Default minimum / recommended values
- Users can:
  - Toggle criteria on/off (e.g., education might be non-essential).
  - Change relevance weights (0–1) to influence scoring.
- The system enforces a **normalized weighting scheme**:
  - Total weight across active criteria = 1.0 (or 100%).
  - If the user changes one weight, others are automatically re-normalized or user is prompted to adjust.

Persistence:

- Finalized criteria configuration is saved as a versioned configuration for the job:
  - `job_id`
  - `criteria_config_vX` (to track changes over time)

5.1.3 Resume Ingestion and Analysis

Goal: Bulk process resumes and produce a ranked list of candidates.

Inputs:

🚀 Powered by Yupp

- Bulk upload of resumes in PDF format (100–1000+ resumes).
- Selected job posting (with associated criteria configuration).

Resume Ingestion Flow:

1. **Bulk Upload:**
   - User selects job and uploads:
     - A single zip file of PDFs, or
     - Multiple PDFs via drag-and-drop.
   - System validates:
     - File type (PDF only for MVP).
     - Size limits per file and per batch.
2. **Parsing:**
   - Each PDF is assigned a unique candidate ID.
   - A PDF parsing library extracts text content.
   - Basic cleanup:
     - Remove headers/footers where possible.
     - Normalize whitespace and bullet points.
3. **Resume Analysis via LLM:**
   - For each resume:
     - The system sends:
       - Resume text.
       - Job description.
       - Criteria configuration (including weights and thresholds).
     - LLM is instructed to:
       - Extract key candidate attributes: years of experience, skills, education, location, domain knowledge.
       - Compute a sub-score per criterion (0–100 or 0–1).
       - Provide a brief justification for each score.
4. **Scoring Logic:**
   - For each candidate:
     - `criterion_score` computed by LLM per criterion.
     - Final resume score:

🚀 Powered by Yupp

$$resume_score = \sum (criterion_score_i \times weight_i)$$

- Threshold checks:
  - If candidate **fails critical threshold** (e.g., fewer than minimum years of experience), the system may:
    - Lower bound their score.
    - Flag as "below minimum requirements".

5. **Ranking & Selection:**
   - Candidates are sorted by `resume_score` descending.
   - Top 10% are automatically flagged as **"Recommended for Interview"**.
   - Others remain visible with their scores.

Outputs:

- For each candidate:
  - Parsed profile (normalized data).
  - Resume score.
  - Criterion-level breakdown.
  - Recommendation flag (recommended / not recommended).
- Stored in DB and accessible via UI.

5.2 Interview Assistance Module

5.2.1 Audio Capture and Transcription

Goal: Capture and transcribe live interviews, distinguishing between interviewer and candidate.

Constraints (MVP):

- Single microphone capturing both parties.
- No separate audio channels per speaker.

Inputs:

- Live audio stream from browser (or desktop client).
- Metadata:

🚀 Powered by Yupp

- Interviewer ID
- Candidate ID (selected from previously screened candidates)
- Job ID

Processing Flow:

1. **Session Initialization:**
   - Interviewer opens the **Interview Assistance Dashboard**.
   - Selects:
     - Job
     - Candidate
   - Starts a new "Interview Session".
2. **Audio Capture:**
   - Browser requests permission for microphone access.
   - Once granted, audio chunks are streamed in near real-time to backend.
3. **Speech-to-Text with Diarization:**
   - Audio is sent to the speech processing service (e.g., Assembly AI or similar).
   - The service:
     - Transcribes speech to text.
     - Performs **speaker diarization** (identifies speaker segments: speaker A, speaker B).
   - System maps:
     - Speaker roles (heuristic):
       - First speaker in first few seconds is assumed to be interviewer.
       - Continues to refine mapping based on talk patterns (optional).
     - Or interviewer can manually set which speaker ID is them in the UI once enough speech is captured.
4. **Real-Time Transcript Updates:**
   - Transcription segments are streamed back to the front-end via WebSocket/SSE.
   - UI displays:
     - Timestamped transcript.
     - Speaker labels: "Interviewer" vs "Candidate".

Outputs:

🚀 Powered by Yupp

- Live transcript visible to interviewer.
- Persistent transcript stored in database after session ends.

5.2.2 Real-Time Interviewer Assistance

Goal: Provide the interviewer with intelligent, contextual support during the interview.

Inputs:

- Real-time transcript (candidate and interviewer utterances).
- Candidate resume and screening insights.
- Job description and criteria configuration.

Features:

1. **Real-Time Transcription Panel**
   - Shows the evolving transcript.
   - Distinguishes speaker roles.
2. **Contextual Analysis Loop**
   - As new candidate responses come in:
     - System batches recent candidate-specific utterances (e.g., last 2–3 turns).
     - Sends to LLM along with:
       - Job requirements.
       - Candidate resume summary/parsed info.
       - Evaluation criteria.
     - LLM analyzes:
       - Which criteria are being addressed.
       - How strong the candidate's response is.
       - Which critical areas remain uncovered.
3. **Suggestions and Guidance:**
   - The LLM returns:
     - **Suggested follow-up questions**
       - E.g., "Can you describe a time you optimized a system for performance?"
     - **Areas to probe**
       - E.g., "Clarify candidate's role in the described project – leadership vs. contributor."

🚀 Powered by Yupp

        - **Contextual insights**
            - E.g., “Candidate claims 5+ years in Python, resume indicated 3 years – consider probing.”
4. **UI Presentation:**
    - A side panel labeled “AI Suggestions” shows:
        - Top 1–3 suggested follow-up questions.
        - Brief notes (“Insights”) explaining why they matter.
    - Suggestions update automatically every few seconds or after each candidate response, with rate-limiting to avoid overload.
5. **Transparency:**
    - This assistance is visible **only to the interviewer**.
    - Candidate is not notified and has no access to AI assistance.

Outputs:

- Stream of suggestions and insights (not stored in full for MVP or stored as logs depending on requirements).
- Enhanced interviewer experience and better coverage of key criteria.

5.2.3 Post-Interview Scorecard Generation

Goal: Summarize the interview into an objective scorecard.

Inputs:

- Full interview transcript (with speaker roles).
- Job description and criteria configuration.
- Candidate resume and resume screening output.
- Optional interviewer notes taken in real-time.

Processing Flow:

1. **Session End:**
    - Interviewer clicks “End Interview” in the UI.
    - System finalizes and stores the complete transcript.
2. **LLM-Based Evaluation:**
    - Backend sends:
        - Entire transcript (or chunked but contextually reconstructed).

🚀 Powered by Yupp

        - Candidate resume summary.
        - Job requirements and criteria with weights.
    - LLM is instructed to:
        - Evaluate candidate responses against each criterion.
        - Compare claims in the interview vs. resume (e.g., inconsistencies).
        - Produce:
            - Criterion scores.
            - Qualitative remarks per criterion.
            - Overall interview performance summary.
            - Strengths and weaknesses.
3. **Scorecard Structure:** Example structure:
    - **Overall Interview Score** (0–100)
    - **Per-Criterion Detail:**
        - Criterion name
        - Score (0–100)
        - Evidence-based explanation referencing transcript.
    - **Behavioral/Soft Skills Assessment** (if part of criteria)
    - **Risk/Concern Flags** (e.g., inconsistent experience dates).
4. **Interviewer Feedback Integration:**
    - After the initial scorecard is generated, the UI prompts the interviewer to add:
        - Overall rating (e.g., “Strong Hire”, “Hire”, “No Hire”).
        - Free-text feedback.
        - Notes on cultural fit or specific observations.
    - Interviewer feedback is:
        - Stored separately.
        - Later incorporated into the final evaluation engine (weighted appropriately).

Outputs:

- A persistent **Interview Scorecard** object, linked to:
    - Candidate
    - Job
    - Interview session

🚀 Powered by Yupp

5.3 Final Candidate Evaluation

Goal: Provide a single, holistic score and summary to help the hiring manager decide.

Inputs:

- Resume screening score and breakdown.
- Interview scorecard and sub-scores.
- Interviewer feedback (subjective rating + comments).
- Job criteria configuration (for weighting).

Aggregation Logic:

1. Define top-level weights such as:
   - Resume score: 30%
   - Interview score: 50%
   - Interviewer subjective rating: 20%
2. **Normalize scores:**
   - Resume score and interview score normalized to 0–100.
   - Convert subjective rating to numeric scale, e.g.:
     - Strong Hire = 100
     - Hire = 80
     - Neutral = 60
     - No Hire = 20
3. **Compute Final Score:**

$$final_score = resume_score \times 0.3 + interview_score \times 0.5 + subjective_score \times 0.2$$

4. **Generate Summary:**
   - LLM produces a short narrative:
     - Major strengths and concerns.
     - Fit with core requirements.
     - Recommendation status: “Recommend progression”, “Borderline”, “Do not progress”.

🚀 Powered by Yupp

Outputs:

- Final score per candidate per job.
- Summary narrative.
- Status recommendation (e.g., “Advance to next round”, “Hold”, “Reject”).

Usage:

- Hiring managers view a table/list of all candidates with:
  - Final score
  - Key indicators (skills fit, interview performance)
  - Quick access to resume and scorecard
- Enables easy comparison and shortlisting.

---

6. Non-Functional Requirements

6.1 Scalability

- Must handle:
  - 100–1000+ resumes per job upload.
  - Multiple concurrent resume processing jobs.
  - Multiple concurrent interview sessions.
- Architecture should allow horizontal scaling of:
  - Worker processes for LLM/resume analysis.
  - Real-time transcription handlers.

6.2 Performance

- Resume processing:
  - Should complete screening of 100 resumes within a reasonable time (e.g., under 30–60 minutes in typical conditions, actual SLAs can be set later).
- Interview assistance:
  - Transcription latency target: < 2–3 seconds from speech to text display.
  - Suggestion latency target: updated within a few seconds of candidate response.

6.3 Security & Privacy

🚀 Powered by Yupp

- All data in transit: HTTPS/TLS.
- Data at rest: encrypted where possible (DB encryption or disk-level).
- Access control:
  - Authentication (e.g., via username/password, SSO if desired later).
  - Role-based authorization:
    - Recruiter
    - Hiring manager
    - Admin
- Data handling:
  - Only authorized users can see candidate data.
  - Logs must avoid storing raw PII excessively where not needed.

(Full regulatory compliance is out of MVP scope, but best practices should be followed.)

6.4 Reliability

- System should handle:
  - Transcription/LLM API failures with retries and graceful fallbacks.
  - Partial failures in resume processing with per-file error reporting.
- Interview sessions:
  - If connection drops, attempt reconnection and resume transcript where possible.

6.5 Usability

- Intuitive UI:
  - Clear job creation and criteria configuration.
  - Simple resume upload workflow.
  - Clear status indicators during processing.
  - Clean, readable scorecards and final evaluation views.

---

7. Technical Architecture

7.1 Suggested Technology Stack

Backend:

🚀 Powered by Yupp

- Language/Framework: Node.js (NestJS / Express) or Python (FastAPI / Django) – team choice.
- Runtime: Containerized (Docker) to support cloud deployment.

Frontend:

- React/Next.js or equivalent SPA framework.
- Real-time communication via WebSocket or Server-Sent Events (SSE).

AI Services:

- **LLM:** Gemini 2.0 Flash (or equivalent):
  - Resume analysis
  - Criteria extraction
  - Interview assistance suggestions
  - Scorecard and final evaluation generation
- **Speech Processing:** Assembly AI (or equivalent):
  - Real-time transcription
  - Speaker diarization

Data Storage:

- Primary DB: PostgreSQL or MySQL for structured data.
- Object storage: S3-compatible for:
  - PDFs
  - Audio recordings (if stored)
  - Raw transcripts (optional, or DB).

PDF Processing:

- Python libraries (e.g., pdfminer, PyPDF2) or Node equivalents (e.g., pdf-parse).

7.2 High-Level Architecture

Main Components:

1. **Frontend Web App**
   - Job and criteria management
   - Resume upload & status
   - Interview dashboard (transcript + suggestions)

🚀 Powered by Yupp

- Results/scorecards view

2. **Backend API**
   - REST/GraphQL endpoints
   - WebSocket endpoints for real-time updates
   - Orchestration of:
     - Resume parsing
     - LLM calls
     - Transcription services
   - Business logic for scoring and evaluation
3. **Worker Services**
   - Background workers for:
     - Bulk resume processing
     - Scorecard generation
   - Communicate via message queue (e.g., RabbitMQ / Redis queue) if necessary.
4. **External Integrations**
   - LLM API
   - Speech-to-text API
5. **Database & Storage**
   - Job, Candidate, InterviewSession, Scorecard, CriteriaConfig tables.
   - File storage for PDFs and optional audio files.

7.3 Data Model (Simplified)

- **Job**
  - id
  - title
  - description
  - criteria_config (JSON)
  - created_at, updated_at
- **Candidate**
  - id
  - name (optional; may be derived from resume)
  - email (optional)

🚀 Powered by Yupp

    - resume_file_path
    - parsed_resume (JSON)
    - created_at, updated_at
- **Application** (relationship between Job & Candidate)
    - id
    - job_id
    - candidate_id
    - resume_score
    - resume_score_breakdown (JSON)
    - status (e.g., “Screened”, “Recommended”, “Rejected”)
- **InterviewSession**
    - id
    - job_id
    - candidate_id
    - interviewer_id
    - start_time, end_time
    - transcript (text or JSON segments)
    - raw_transcript_file_path (optional)
    - diarization_metadata (JSON)
- **InterviewScorecard**
    - id
    - interview_session_id
    - overall_score
    - criterion_scores (JSON)
    - summary
    - strengths
    - weaknesses
- **InterviewerFeedback**
    - id
    - interview_session_id
    - rating (enumeration, e.g., “Strong Hire”, “Hire”, “No Hire”)
    - numeric_rating (0–100)
    - comments

🚀 Powered by Yupp

- **FinalEvaluation**
  - id
  - job_id
  - candidate_id
  - resume_score
  - interview_score
  - subjective_score
  - final_score
  - recommendation_status
  - created_at

---

8. User Interfaces & Workflows

8.1 Job Creation & Criteria Configuration UI

Pages:

1. **Create Job**
   - Fields: Job title, department, seniority (optional).
   - Text area for job description.
   - Button: “Generate Criteria”.
   - On submit: calls backend → LLM → shows generated criteria.
2. **Criteria Configuration**
   - Table of criteria:
     - Name, Description, Threshold, Weight, Required (yes/no)
   - Controls:
     - Numeric inputs for thresholds and weights.
     - Checkbox/toggle for Required criterion.
   - Button: “Save & Activate”.

8.2 Resume Upload & Screening UI

Pages:

1. **Job Detail / Resumes**

🚀 Powered by Yupp

- Section: Upload Resumes
  - Drag-and-drop area or file selector, multiple PDFs.
  - Shows upload progress.
- Section: Processing Status
  - Count: “Resumes queued / processing / completed”.
- After processing:
  - Table:
    - Candidate name / ID
    - Resume score
    - Top skills
    - Status (Recommended / Not recommended)
  - Sorting and filtering by score.

8.3 Interview Assistance Dashboard

Layout:

- **Left Panel:**
  - Job and candidate summary.
  - Key criteria, with quick indicators (met/partially/unmet based on resume).
- **Center Panel:**
  - Real-time transcript:
    - Messages grouped by timestamp.
    - Speaker labels (“Interviewer”, “Candidate”).
- **Right Panel:**
  - “AI Suggestions”:
    - Top 1–3 follow-up questions.
    - Bullet list of insights and potential gaps.
  - Controls:
    - Start/Stop recording.
    - Button: “Generate Scorecard” (enabled after interview ends).

8.4 Scorecard & Final Evaluation UI

Scorecard View:

🚀 Powered by Yupp

- Header:
  - Candidate + Job
  - Date of interview
- Overall Interview Score
- Per-criterion scores + short explanations.
- Strengths & weaknesses.

Interviewer Feedback Form:

- Radio buttons: Strong Hire / Hire / Neutral / No Hire
- Text area for free-form feedback.
- Submit button: “Save Feedback & Update Final Score”.

Final Evaluation List:

- For each candidate:
  - Final score
  - Resume score
  - Interview score
  - Subjective rating
  - Recommendation (Advance / Hold / Reject)
- Sorting, filtering, and detail drill-down to scorecard and transcript.

---

9. Assumptions & Constraints (Re-stated and Operationalized)

9.1 Assumptions

- Resumes are provided only in **PDF** format.
- Interviews use **one shared microphone** for interviewer and candidate.
- Internet connectivity is available for:
  - LLM calls
  - Speech-to-text API calls
- Users (recruiters, hiring managers) have **basic technical proficiency**.

9.2 MVP Constraints

🚀 Powered by Yupp

- No ATS integration (imports/exports only via manual upload/download).
- No automated scheduling or calendar integration.
- Only **first-round interviews** supported; no multi-round pipeline tracking.
- No video interviewing (audio only).
- No advanced analytics/dashboard (simple lists and summaries only).
- No candidate-facing portal (all internal).

---

10. Success Criteria (How We'll Measure)

The MVP is considered successful if:

- System processes and ranks **≥ 100 resumes per job** without failure.
- Speaker diarization achieves **> 85% accuracy** in distinguishing interviewer vs. candidate.
- Interviewer assistance suggestions are rated "relevant/useful" by pilot users.
- Scorecards:
  - Cover all defined evaluation factors.
  - Are judged "clear and useful" by pilot users.
- Overall:
  - Time-to-shortlist reduced by at least **50%** versus manual process for pilot roles.
  - Positive UX feedback from recruiters and hiring managers.

---

11. Testing & Acceptance Criteria

11.1 Functional Testing

- **Resume Processing Tests:**
  - Upload valid PDFs.
  - Upload corrupted/unsupported files → error handling.
  - Verify criteria extraction and scoring.
- **Interview Tests:**
  - Confirm real-time transcription and diarization.

🚀 Powered by Yupp

    - Validate that assistance suggestions are updated and contextually relevant.
- **Scorecard & Evaluation Tests:**
    - Confirm scorecard generation for different roles.
    - Verify final score calculations with different weight combinations.

11.2 Performance Benchmarks

- Batch of 100 resumes:
    - Successfully processed and ranked.
- Real-time transcription:

🚀 Powered by Yupp